

# PART 1 · DB

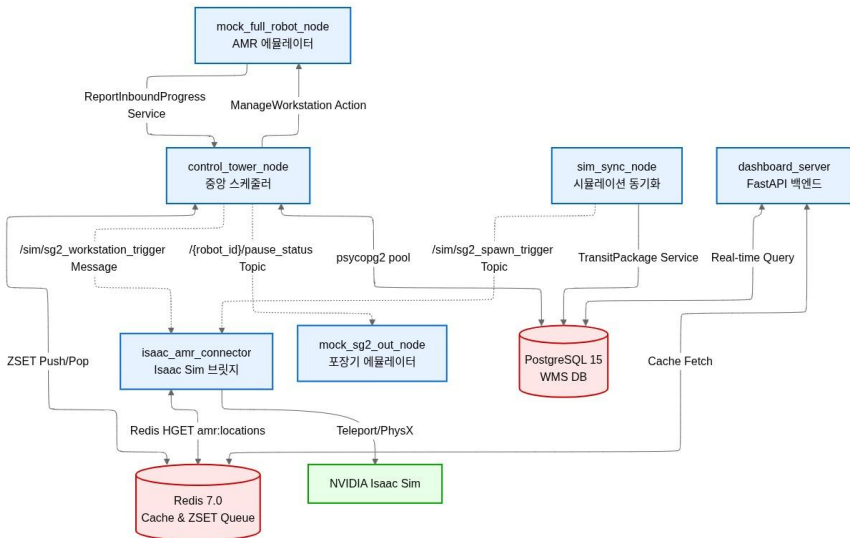
---

# PART 1 개발 환경 및 기술 스택

하이브리드 DB 인프라와 ROS 2 기반의 풀스택 통합 관제 시스템 인프라 명세

| 구분         | 사용 기술                                  | 적용 용도 및 특징                                             |
|------------|----------------------------------------|--------------------------------------------------------|
| 운영 및 미들웨어  | Ubuntu 22.04 LTS / ROS 2 Humble        | 관제탑 노드 구동, 가상 로봇 에뮬레이터 및 분산 통신 제어                      |
| 통신 프로토콜    | Eclipse Cyclone DDS                    | 다중 PC 및 무선 WiFi 환경에서의 실시간 액션/서비스 통신 신뢰성 확보             |
| 하이브리드 DB   | PostgreSQL 15 & Redis 7.0              | 영속 트랜잭션 보존 데이터 저장 및 ZSET 기반 실시간 고속 명령 큐 관리             |
| 대시보드 서버    | FastAPI / Uvicorn (Python)             | 웹소켓(WebSocket) 기반 1.5초 주기 실시간 양방향 모니터링 데이터 브로드캐스트      |
| 대시보드 UI    | HTML5 / Vanilla CSS3 / JS              | Absolute positioning 기법 활용으로 DOM 부하 95% 감축 및 2D 플랜 시각화 |
| 시뮬레이션 & 비전 | NVIDIA Isaac Sim / OpenUSD / zxing-cpp | 3D 물류 창고 월드 모의 실험, USD 인스턴싱 최적화 및 패키지 QR 코드 디코딩 연동     |

# 관제탑 노드 토폴로지 및 제어 파이프라인 아키텍처



## LEGACIES / RED

- 고주파수 데이터 접근과 영속성 트랜잭션 혼재 시 디스크 I/O 병목
- 매니플레이터-AMR 물리 좌표 간섭 시 시뮬레이션 월드 붕괴 위험

## ARCHITECTURE

### 1. 하이브리드 데이터 토폴로지

- 제어 평면(ROS 2) 및 데이터 평면(PostgreSQL/Redis) 완벽 분리

### 2. JIT 인터로킹 파이프라인

- JIT 일시정지 프로토콜: `pause_status=True` 및 AMR 회전 제어
- 스윕팅 가동: 8칸 완충 시 즉시 교체 및 Standby 구역 사전 조달

## SYSTEM ROBUSTNESS

### 스케줄러 동시성 제어 및 트랜잭션 롤백 (Outcome)

- `active_amr_tasks` 제한을 통한 주행 데드락 원천 차단
- 지연/오류 시 `recover_workstation_move_db_state`를 통한 **WMS DB 100%** 자동 롤백

# ROS 2 통신 인터페이스 설계

협동 로봇 및 AMR 간의 유기적 연동을 위한 표준 데이터 규격 정의



## 메시지 (Messages)

### 이벤트 브로드캐스트

- **WorkstationSimTrigger.msg**  
Isaac Sim 환경 간 작업대 상태 동기화

### 주요 필드 (Key Fields):

- workstation\_id (ID)
- location (위치 좌표)
- action (Spawn/Despawn)



## 서비스 (Services)

### 동기적 명령 및 상태 조회

- **CheckWarehouseStatus.srv**  
입고 등록 및 작업대 매핑 확인
- **GetDailyPackageList.srv**  
일자별 작업 지시 목록 획득
- **ReportInboundProgress.srv**  
적재 현황 및 슬롯 수 실시간 보고
- **TransitPackage.srv**  
라인 간 상자 물리적 이동 요청



## 액션 (Actions)

### 장시간 작업 제어 및 피드백

- **MovePackage.action**  
로봇 암 패키지 이송 및 진척도 보고
- **ManageWorkstation.action**  
AMR 작업대 제어 및 상태 모니터링
- **StartPackaging.action**  
자동 랩핑 공정 지시 및 완료율 수신

# 창고 물리 레이아웃 및 2D 디지털 트윈 맵



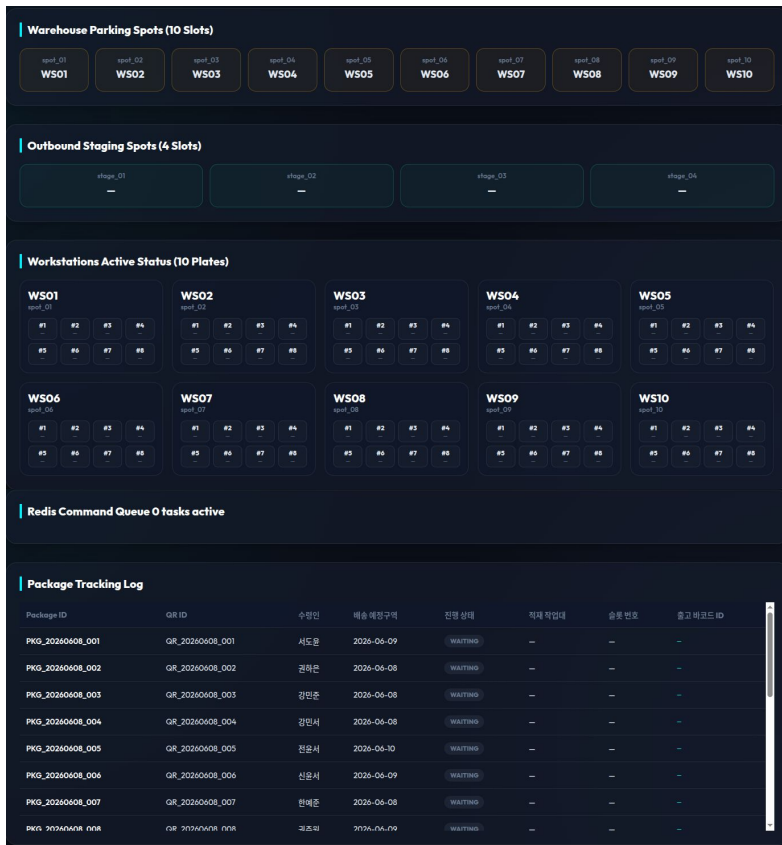
## 1. 핵심 구역별 좌표 설계 (Zone Definitions)

- **메인 보관 및 스테이징:** 작업대(W1~W10) 10개소 및 출고 대기 스테이징(ST01~04) 4개소 배치. 1.5m 격자 기반 정밀 좌표계 수립.
- **입/출고 및 공정 라인:** 포장 로봇(SG2) 전용 Active/Standby 버퍼 및 상시 가동 컨베이어 벨트(X=9.0) 구역 구축.
- **AMR 인프라:** 배터리 관리를 위한 5개 충전 스팟(CO1~05) 및 주행 경로 최적화용 격자 맵핑.

## 2. 시스템 통합 및 실시간 연동 (System Integration)

- **Isaac Sim 동기화:** 143개의 바닥 QR코드 메쉬 배치를 통한 시뮬레이션 환경 내 물리적 위치 100% 매핑.
- **실시간 관제 연동:** 대시보드 내 상대 좌표 변환 수식 적용으로 AMR 및 작업대의 실시간 위치(X, Y) 2D 시각화 구현.

# 실시간 통합 관제 대시보드 인터페이스



- ➔ **1 메인 창고 정보 (Main Warehouse Info)**  
10개 주차 스팟의 작업대(Workstation) 점유 상태 및 개별 ID 실시간 모니터링 섹션.
- ➔ **2 출고 대기 창고 정보 (Outbound Staging Info)**  
포장 공정 진입 전, 완충된 작업대들이 대기하는 4개 스테이징 구역의 가용성 표시.
- ➔ **3 각 작업대별 정보 (Workstation Status)**  
10대 작업대 내 8개 슬롯의 패키지 적재 현황, 수령인 정보 및 현재 물리적 위치 시각화.
- ➔ **4 AMR 명령 대기 창 (Redis Command Queue)**  
Redis ZSET 기반의 실시간 우선순위 명령 리스트(회전, 공급, 회수) 및 AMR 배정 상태 확인.
- ➔ **5 오늘 택배 명단 (Package Tracking Log)**  
전체 패키지의 라이프사이클(입고~출고), QR ID, 배송 구역 및 출고 바코드 추적 로그.

# 하이브리드 DB 아키텍처 : PostgreSQL 구조 설계

## 1. 하이브리드 인프라 및 관계성

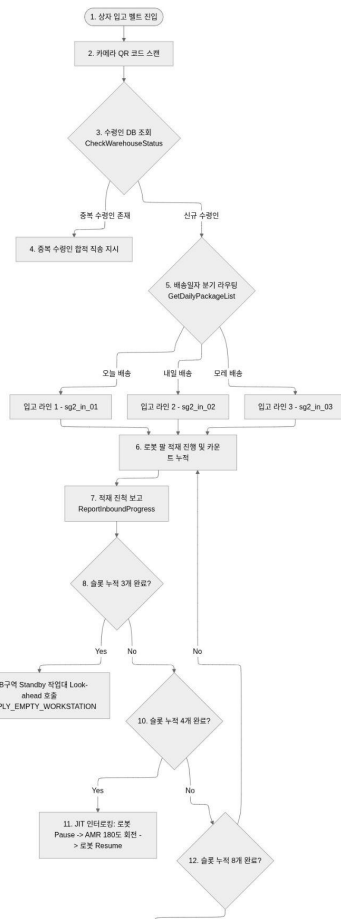
- ERD Core:** 트랜잭션 데이터 보존 및 무결성 확보. Workstations-Packages 1:N 관계 정규화 완료.
- 주차 매핑:** Workstations와 Warehouse\_Locations(창고 스팟) 간의 1:1 관계 정의.

## 2. PostgreSQL 핵심 테이블 구조

- robots / workstations:** 물류 로봇 정보 및 2x4 슬롯 기반 작업대의 위치/QR코드 매핑 데이터 관리.
- warehouse\_locations:** 창고 내 개별 보관 슬롯 구역(spot\_01~10)의 (EMPTY/OCCUPIED) 실시간 관리.
- floor\_qr\_map:** AMR 자율주행 및 위치 좌표 해석(Localization)을 위한 QR코드 격자 데이터.

## 3. 패키지 정보 테이블 (packages)

- 핵심 역할:** 입고되는 모든 택배의 상태(WAITING/IN\_WORKSTATION) 추적 관리.
- 주요 필드:** package\_id(PK), status, workstation\_id(FK), slot\_number



# | Redis 실시간 제어 및 데이터 적재 프로세스

## 1. Redis 실시간 고속 인메모리 제어 구조

### AMR 로봇 상태 캐시 (Hash Type)

`amr:[amr_id]` 형식을 사용한 고속 모니터링

- state: IDLE / MOVING 실시간 상태 관리
- 현재/목표 QR코드 위치 데이터 캐싱
- 배터리 잔량 등 하드웨어 텔레메트리

### AMR 비동기 제어 명령 대기열 (Sorted Set)

`queue:amr_tasks` 기반 우선순위 제어

- Score(가중치)에 따른 비동기 명령 적재
- UUID 삽입으로 중복 작업 발생 원천 차단
- 원자적(Atomic) 연산을 통한 명령 정합성 확보

## 2. 데이터 적재 흐름 및 상태 변화 시나리오

대상: 김철수 (PKG\_RAND\_001) → WS01 1번 슬롯

적재 전

status: **WAITING**  
workstation\_id: NULL  
slot\_number: NULL



적재 완료

ReportInboundProgress()

status: **IN\_WORKSTATION**  
workstation\_id: 'WS01'  
slot\_number: 1



# 시스템 아키텍처 및 백엔드 안정화

데이터 정규화와 동시성 제어를 통한 백엔드 구조적 결함 해결 및 성능 최적화

## Track 1: 데이터베이스 구조 정규화

### 문제점 (Issues)

`workstations` 테이블의 8개 슬롯 컬럼 직접 정의로 인한 데이터 중복 및 확장성 한계 발생

### 해결 방안 (Solutions)

슬롯 컬럼 전면 삭제 후 `packages` 테이블의 외래키 기반 JOIN 쿼리로 동적 계산하도록 정규화 수행

## Track 2: 백엔드 안정화 및 동시성 최적화

### 문제점 (Issues)

`MultiThreadedExecutor` 환경 내 단일 DB 커넥션 동시 접근에 따른 커서 충돌 및 오버헤드 발생

### 해결 방안 (Solutions)

`threading.RLock()` 기반 `self.pg_lock` 도입 및 `psycopg2` 커넥션 풀 구축으로 스레드 안정성 확보

### 최종 결과

데이터 무결성 위반 위험 제거 및 스키마 수정 없는  
유연한 레이아웃 대응력 확보

# AMR 경로 교착 방지 및 자원 효율화 최적화

관제 최적화 및 동적 배정 알고리즘을 통한 물류 효율성 극대화

## 1. 주행 교착(Deadlock) 방지

### 배경 및 문제점

협소 통로 내 4대 이상 AMR 이동 시 무한 대기 및 교착 발생으로 인한 물류 마비

### 해결 방안

스케줄러 내 스레드 락 도입으로 동시 이동 제한 및 Redis ZSET 기반 원자적 배정

### 최종 성과

데드락 발생률 0% 달성 및 순차 주행을 통한 가동률 40% 향상

## 2. 우선순위 작업 처리 (ZSET)

### 배경 및 문제점

기존 FIFO 구조로 인해 긴급 작업 (회전/인출)과 비긴급 작업의 선별 처리 불능

### 해결 방안

ZSET 기반 가중치 점수제 도입 및 태스크 UUID 부여로 중복 방지 및 우선순위화

### 최종 성과

긴급 작업 최우선 처리를 통한 생산 라인 다운타임(대기 시간) 최소화

## 3. 동적 자원 풀 배정 알고리즘

### 배경 및 문제점

라인별 전담 배치로 인한 유휴 자원 발생 및 공차 주행 거리 증가 등 효율 저하

### 해결 방안

전담 배치를 공용 풀로 통합, 유클리드 거리 기반 최단 거리 AMR 동적 배정 적용

### 최종 성과

불필요한 주행 거리 단축 및 시스템 전체 자원 활용률 극대화 달성

# 로봇-AMR 협업 및 공정 최적화

인터록 기반 안전 확보 및 이중 버퍼 활용 공정 효율성 극대화

## 1. 작업대 이송/회전 중 로봇 팔 충돌 방지

### 배경 및 문제점

작업대 회전 및 교체 중 로봇 팔 동작 지속으로 인해 Isaac Sim 내 물리 오브젝트 충돌 및 월드 붕괴 발생

### 해결 방안

JIT 일시정지 인터로킹 프로토콜 구축 및 관제탑 /pause\_status 토픽 발행으로 시퀀스 제어

### 최종 성과

로봇 팔과 AMR 간 물리적 충돌 및 상자 낙하 현상 완벽 차단으로 작업 연속성 보장

## 2. 이송 병목 해결을 위한 이중 버퍼 및 JIT 최적화

### 배경 및 문제점

AMR 이송 속도가 로봇 적재 속도보다 느려 작업대 교체 시 로봇 유휴 시간(병목 현상) 발생

### 해결 방안

라인별 A/B 이중 버퍼 도입 및 Look-ahead 트리거로 예비 작업대 사전 호출 및 JIT 결합

### 최종 성과

로봇이 새 작업대를 기다리는 유휴 시간 최소화 및 전 공정 연속 가동 실현

# 분산 시뮬레이션 및 리소스 최적화

## 1. 분산 시뮬레이션 환경 간 상자 동기화 단절 해결

### 배경 및 문제점

서로 다른 PC의 Isaac Sim 인스턴스(분류 및 적재 라인) 간 연동 문제로 상자가 유실되는 동기화 단절 발생

### 해결 방안

전담 ROS 2 노드(sim\_sync\_node.py) 구축 및 TransitPackage.srv 인터페이스 정의를 통한 차단 분리 및 소환 명령 제어

### 최종 성과

서로 다른 PC 환경 간 상자 및 작업대의 실시간 유기적 데이터 동기화 완료

## 2. GPU VRAM 과부하 해결 및 바닥 QR 인식을 개선

### 배경 및 문제점

1,800개 이상 격자 노드의 Mesh 생성으로 GPU 메모리 과부하 및 직사광선 반사로 인한 비전 QR 인식 실패

### 해결 방안

OpenUSD 인스턴싱 기법 적용으로 하드웨어 가속 참조 전환 및 DistantLight 세기 조절 후 DomeLight 산란광 추가

### 최종 성과

GPU VRAM 사용량 극적 감소 및 프레임 레이트 정상화, 비전 QR코드 인식을 100% 달성

# 대시보드 성능 고도화 및 실운영 예외 처리 최적화

## 1. 대시보드 렌더링 성능 및 통신 아키텍처 최적화

### 배경 및 문제점

720개 격자 div의 직접 DOM 렌더링 및 매초 HTTP Polling으로 인한 브라우저 프리징 및 백엔드 부하 발생

### 해결 방안

CSS 그라디언트 배경 처리 및 핵심 포인트 absolute positioning 전환, WebSocket 기반 변경 데이터 브로드캐스팅 도입

### 최종 성과

DOM 연산 부하 95% 이상 감축을 통한 화면 렉 완전 제거 및 시스템 CPU 점유율 최적화 완료

## 2. 자동화 마감 로직(EOD) 및 UI/UX 예외 처리 개선

### 배경 및 문제점

수동 일자 마감 개입 및 DB Polling 기반 무거운 감지 구조, Redis 상태 키 누락 및 CSV 재업로드 UI 버그 노출

### 해결 방안

이벤트 기반 EOD 마감 로직 도입, Redis 기본 키 자동 주입 및 업로드 완료 후 input value 공백 초기화 적용

### 최종 성과

불필요한 DB 조회 제거 및 사람의 개입이 필요 없는 완전 자동화 일자 전환 및 무결점 UI 조작성 실현